9) | Lecture 17 | B Tree |

08 September 2026 10:57

(2-4) Tree

All the leaves are at the same level.

Each node should have at most three and at least one key.

Each node should have at most four and at least two children.

BST - O(h) can O(n)

AVL - O(log n)

Red Black Tree O(log n)

(2-4) tree

Construction RB and (2-4) tree

Search Equivalent

Insertion

deletion

Slide Credit : Yael Moses, IDC Herzliya

Animated Video B-Tree example



B-Trees

A B-tree is an M-way search tree with two properties :

1. It is perfectly balanced: every leaf node is at the same depth
2. Every internal node other than the root, is at least half-full, i.e. $M/2-1 \le \#keys \le M-1$
3. Every internal node with k keys has $k+1$ non-null children

all leaves are at same level

For simplicity we consider M even and we use t=M/2:

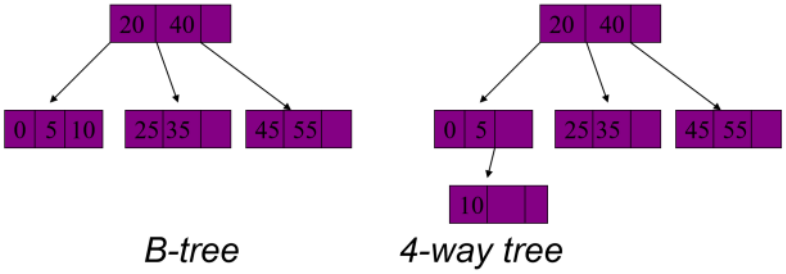
2.* Every internal node other than the root is at least half-full, i.e. $t-1 \le \#keys \le 2t-1$, $t \le \#children \le 2t$

∀ internal node

$t-1 \le \#keys \le 2t-1$

$t \le \#children \le 2t$

Example: a 4-way B-tree



B-tree *4-way tree*

B-tree
1. It is perfectly balanced: every leaf node is at the same depth.
2. Every node, except maybe the root, is at least half-full
   $t-1 \le \#keys \le 2t-1$
3. Every internal node with k keys has $k+1$ non-null children

$K_1 \ldots K_r$ — One node

$t-1 \le r \le 2t-1$

B-tree Height

Claim: any B-tree with n keys, height h and minimum degree t satisfies:

$$h \le \log_t \frac{n+1}{2}$$

$t-1 \le \#keys \le 2t-1$

$t \le \#children \le 2t$

Proof:

- The minimum number of KEYS for a tree with height h is obtained when:
  - The root contains one key
  - All other nodes contain $t-1$ keys

min no. of children for any internal node = t

min no. of children for any internal node = t

min of keys " " := (t-1)

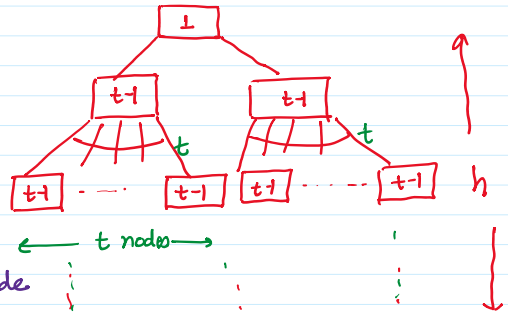
$$n \geq 1 + \left(2 + 2t + 2t^2 + 2t^3 \cdots 2t^{(h-2)}\right)(t-1)$$

(no. of nodes; no. of keys per node)

$$= 1 + 2\left[1 + t + t^2 \cdots + t^{h-2}\right)(t-1)$$

$$n \geq 1 + 2\,\frac{(t^{h-1} - 1)}{(t-1)}\,(t-1) \Rightarrow t^{(h-1)} \leq \left(\frac{n+1}{2}\right)$$

$$\Rightarrow \boxed{h \leq 1 + \log_t\left(\frac{n+1}{2}\right)}$$



B-Tree: Insert X

1. As in *M-way* tree find the leaf node to which *X* should be added
2. Add *X* to this node in the appropriate place among the values already there
   (there are no subtrees to worry about)
3. Number of values in the node after adding the key:
   - Fewer than *2t-1:* done
   - Equal to *2t: overflowed*
4. Fix overflowed node

$(t-1) \leq$ # keys $\leq (2t-1)$

$K_1 \cdots K_r$ r # of keys

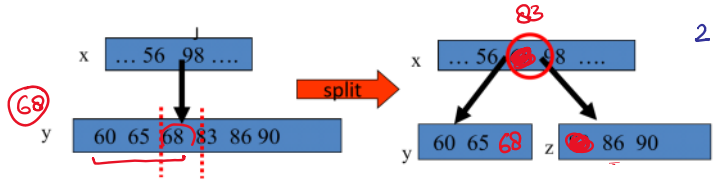
$K_1 < K_2 \cdots < K_r$

Fix an Overflowed

1. Split the node into three parts, *M=2t*:
   - Left: the first *t* values, become a left child node
   - Middle: the middle value at position *t*, goes up to parent
   - Right: the last *t-1* values, become a right child node
2. Continue with the parent:
   1. Until no overflow occurs in the parent
   2. If the root overflows, split it too, and create a new root node

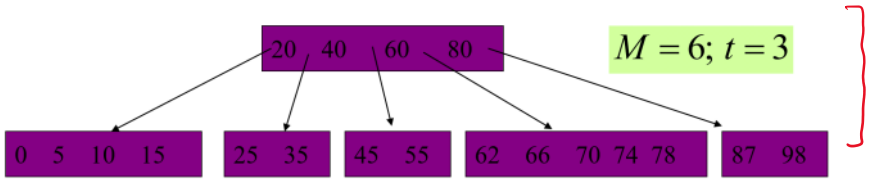
t=3

$2 \leq$ # keys ≤ 5



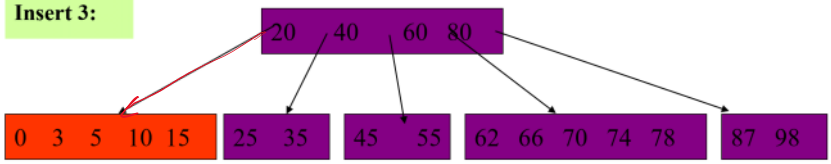
2≤ # keys ≤5

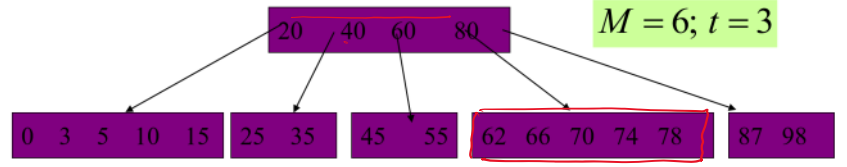
Insert example



$M = 6; t = 3$

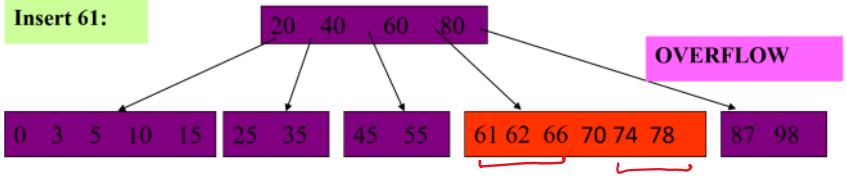
Insert 3:





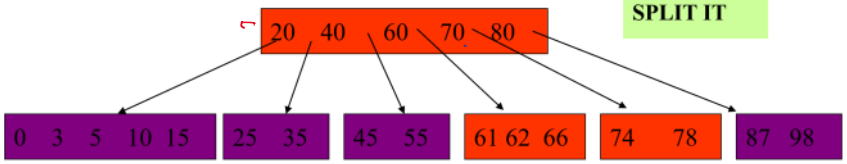
$M = 6; t = 3$

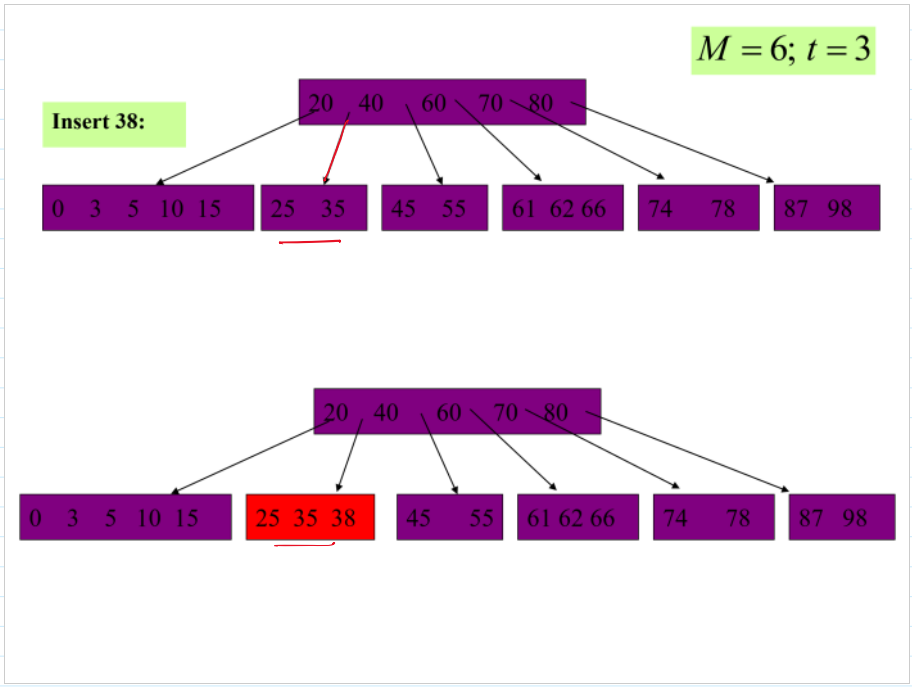
Insert 61:

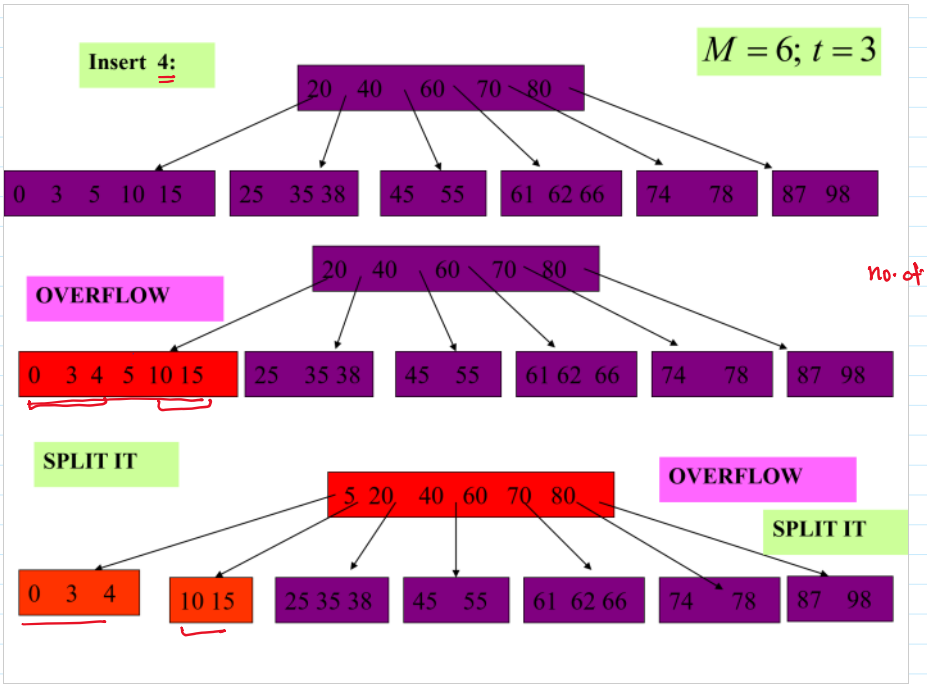


OVERFLOW

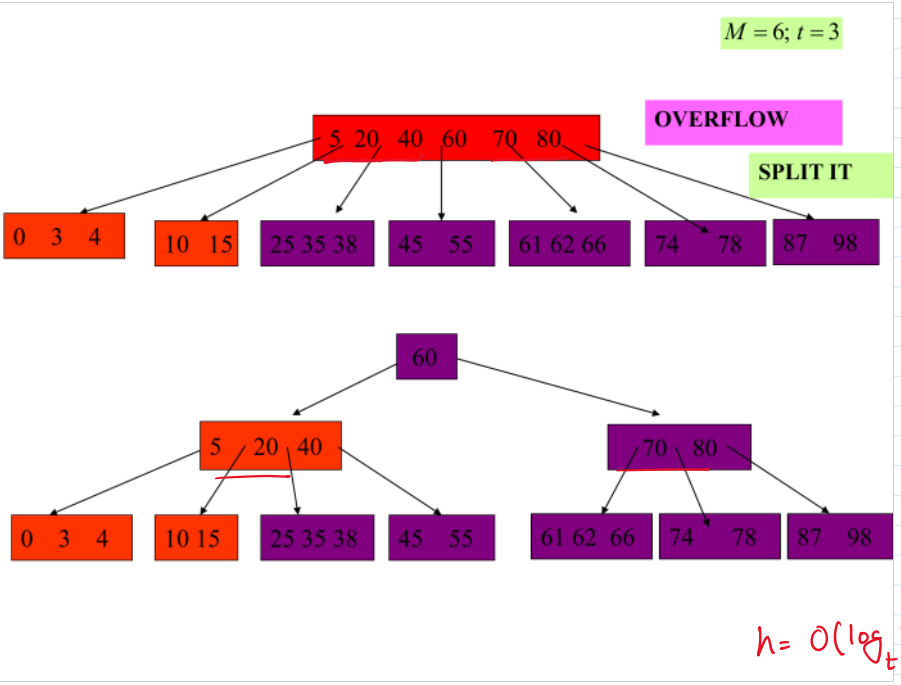
SPLIT IT




$M = 6; t = 3$
Insert 38:
20 40 60 70 80
0 3 5 10 15
25 35
45 55
61 62 66
74 78
87 98
20 40 60 70 80
0 3 5 10 15
25 35 38
45 55
61 62 66
74 78
87 98


$M = 6; t = 3$
Insert 4:
20 40 60 70 80
0 3 5 10 15
25 35 38
45 55
61 62 66
74 78
87 98
OVERFLOW
20 40 60 70 80
0 3 4 5 10 15
25 35 38
45 55
61 62 66
74 78
87 98
SPLIT IT
OVERFLOW
5 20 40 60 70 80
SPLIT IT
0 3 4
10 15
25 35 38
45 55
61 62 66
74 78
87 98

no. of keys = 6



$h = O(\log_t \frac{n+1}{2})$

Complexity Insert

- Inserting a key into a B-tree of height *h* is done in a single pass down the tree and a single pass up the tree

Complexity: $O(h) = O(\log_t n)$

- $(t-1) \leq$ # keys $\leq (2t-1)$
- Ordering among key should be maintained

① The key we want to delete is a leaf node

② The key we want to delete is an internal node

Case 2.1 if no. of keys is $> (t-1)$ then delete that key

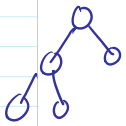
Case 2.2 if no. of keys $< (t-1)$

Case 2.2.1 - if sibling has at least t keys, borrow from sibling

Case 2.2.2 if sibling doesn't have $(t-1)$ keys then merge with sibling and one keys from parent come down.

B-Tree: Delete X

- *Delete as in M-way tree*
- *A problem:*
  - might cause *underflow*: the number of keys remain in a node < t-1



log n

n # of keys

	RB Tree	BTree
Search	$O(\log n)$	$O(\log_t n)$
Insertion	"	"
deletion	"	"

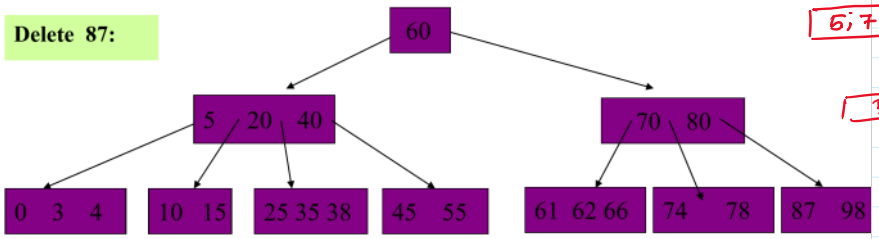
Recall: The root should have at least 1 value in it, and all other nodes should have at least t-1 values in them

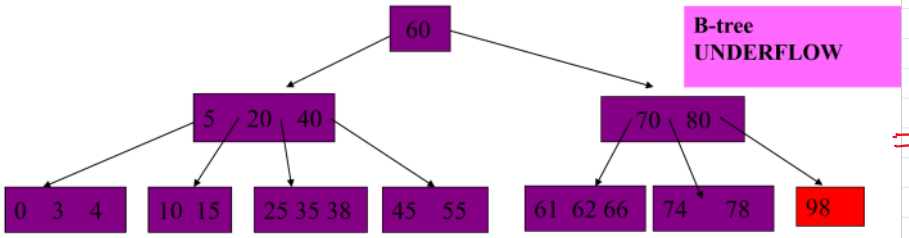
Recall: The root should have at least 1 value in it, and all other nodes should have at least t-1 values in them

$M = 6; t = 3$

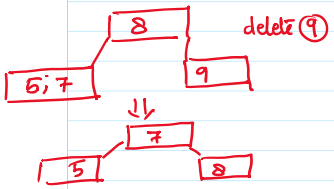
2 ≤ # of keys ≤ 5

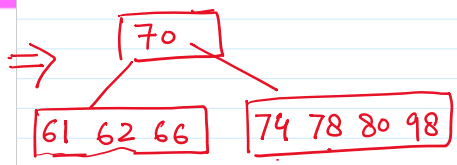
Underflow Example





B-tree UNDERFLOW





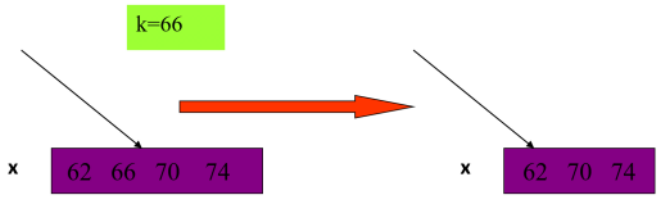
B-Tree: Delete X,k

- *Delete as in M-way tree*
- *A problem:*
  - might cause *underflow*: the number of keys remain in a node < t-1
- Solution:
  - make sure a node that is visited has at least t instead of t-1 keys.
  - If it doesn't have k
    - (1) either take from sibling via a rotate, or
    - (2) merge with the parent
  - If it does have k
    - See next slides

Recall: The root should have at least 1 value in it, and all other nodes should have at least t-1 (at most 2t-1) values in them

B-Tree-Delete(x, k)

1st case: *k* is in *x* and *x* is a *leaf* → delete *k*



How many keys are left?

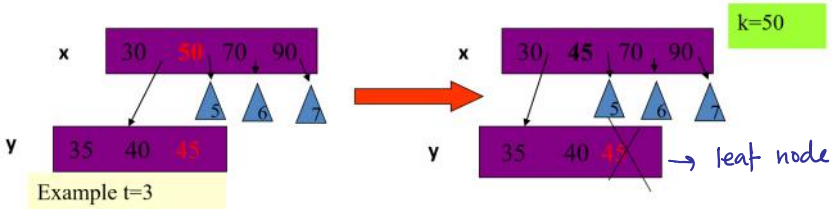
Example t=3

2nd case: *k* in the internal node *x*, *y* and z are the preceding and succeeding nodes of the key k∈ *x*

a. If *y* has at least *t* keys:

- Replace *k* in *x* *k'* ∈ *y*, where *k'* is the predecessor of *k* in *y*
- Delete *k'* recursively

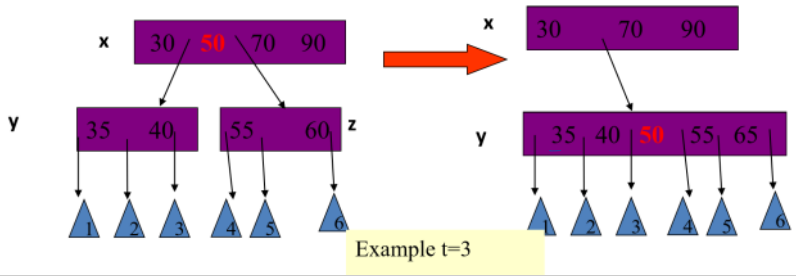
b. Similar check for successor case



Example t=3

2nd case cont.:

c. Both a and b are not satisfied: y and z have *t-1* keys
   - Merge the two children, y and z
   - Recursively delete *k* from the merged cell



Example t=3

Questions

- When does the height of the tree shrink?
- Why do we need the number of keys to be at least *t* and not *t-1* when we proceed down in the tree?

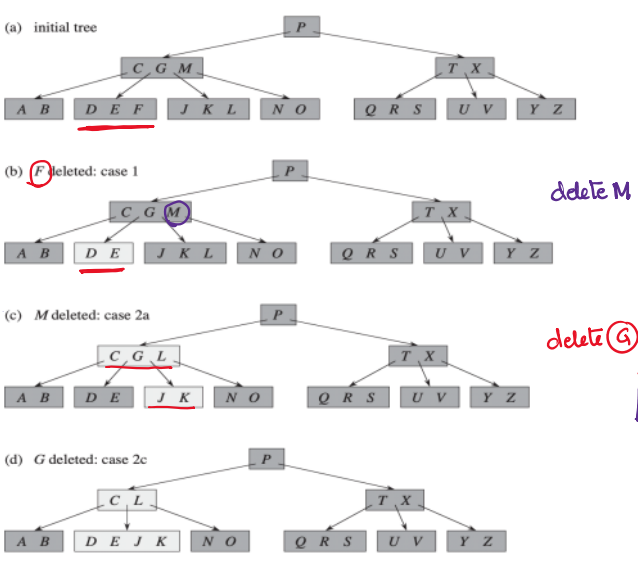


Figure 18.8 Deleting keys from a B-tree. The minimum degree for this B-tree is $t = 3$, so a node (other than the root) cannot have fewer than 2 keys. Nodes that are modified are lightly shaded. **(a)** The B-tree of Figure 18.7(e). **(b)** Deletion of F. This is case 1: simple deletion from a leaf. **(c)** Deletion of M. This is case 2a: the predecessor L of M moves up to take M's position. **(d)** Deletion of G. This is case 2c: we push G down to make node $DEGJK$ and then delete G from this leaf

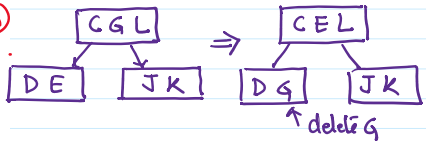
A<B<C
t=3 2≤ # keys ≤5

delete M

swap with pred & delete



delete G



merge with sibling

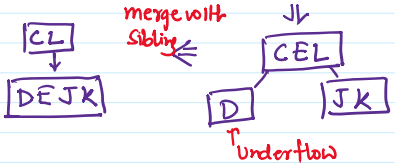


Figure 18.8 Deleting keys from a B-tree. The minimum degree for this B-tree is $t = 3$, so a node (other than the root) cannot have fewer than 2 keys. Nodes that are modified are lightly shaded. **(a)** The B-tree of Figure 18.7(e). **(b)** Deletion of F. This is case 1: simple deletion from a leaf. **(c)** Deletion of M. This is case 2a: the predecessor L of M moves up to take M's position. **(d)** Deletion of G. This is case 2c: we push G down to make node $DEGJK$ and then delete G from this leaf (case 1).

Underflow

Delete Complexity

- Basically downward pass:
  - Most of the keys are in the leaves – one downward pass
  - When deleting a key in internal node – may have to go one step up to replace the key with its predecessor or successor

Complexity $O(h) = O(\log_t n)$

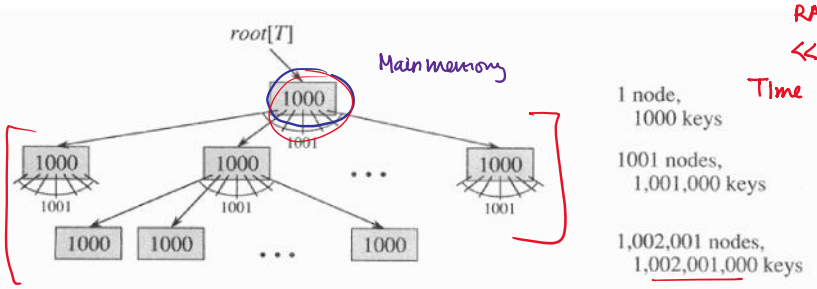
Run Time Analysis of B-Tree Operations

- For a B-Tree of order *M=2t*
  - #keys in internal node: *M-1*
  - #children of internal node: between *M/2* and *M*
  - ➔ Depth of B-Tree storing *n* items is *O(log $_{M/2}$ N)*
- Find run time is:
  - *O(log M)* to binary search which branch to take at each node, since *M* is constant it is *O(1).*
  - Total time to find an item is *O(h*log M) = O(log n)*
- Insert & Delete
  - Similar to find but update a node may take : *O(M)=O(1)*

Note: if M is >32 it worth using binary search at each node

A typical B-Tree

Time req for fetching data from RAM << Time req for fetching data from HDD



HD

Figure 18.3 A B-tree of height 2 containing over one billion keys. Each internal node and leaf contains 1000 keys. There are 1001 nodes at depth 1 and over one million leaves at depth 2. Shown inside each node x is $n[x]$, the number of keys in x.

Copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

Motivation

- Large differences between time access to disk, cash memory and core memory
- Minimize expensive access (e.g., disk access)
- B-tree: Dynamic sets that is optimized for disks

Disk Based Data Structures

- So far search trees were limited to main memory structures
  - Assumption: the dataset organized in a search tree fits in main memory (including the tree overhead)
- Counter-example: transaction data of a bank > 1 GB per day
  - use secondary storage media (punch cards, hard disks, magnetic tapes, etc.)
- Consequence: make a search tree structure secondary-storage-enabled

Hard Disks

- **Large amounts of storage, but slow access!**
- Identifying a page takes a long time (seek time plus rotational delay – 5-10ms), reading it is fast
  - It pays off to read or write data in **pages** (or blocks) of 2-16 Kb in size.

